

CSE 309: Theory of Automata

Decidability Theory

Shahid Hussain

shahidhussain@iba.edu.pk

April 2024

School of Mathematics and Computer Science
Institute of Business Administration

Decidable and Recognizable

Decidable and Recognizable

- For a given Turing Machine M , three outcomes are possible
- The machine may *accept*, *reject*, or *loop*
- When M goes in “loop” it means it does not halt

Turing-Recognizable

A language L is called *Turing-recognizable* if some Turing machine recognizes it. L is also called *recursively enumerable language*.

Decidable and Recognizable

- For a given Turing Machine M , three outcomes are possible
- The machine may *accept*, *reject*, or *loop*
- When M goes in “loop” it means it does not halt

Turing-Recognizable

A language L is called *Turing-recognizable* if some Turing machine recognizes it. L is also called *recursively enumerable language*.

Turing Decidable

A language L is called *Turing-decidable* if some Turing machine decides it. L is also called *recursive language*.

- A machine is called *decider* if halts on every input

Decidability for Regular Languages

- Consider **the acceptance problem** for DFAs of testing whether a particular deterministic finite automataon accepts a given string, lets call this language A_{DFA}

$$A_{\text{DFA}} = \{\langle B, w \rangle : B \text{ is a DFA that accepts input string } w\}$$

- The problem of testing whether a DFA B accepts an input string w is the same as the problem of testing whether $\langle B, w \rangle$ is a member of language A_{DFA}

- Consider **the acceptance problem** for DFAs of testing whether a particular deterministic finite automataon accepts a given string, lets call this language A_{DFA}

$$A_{\text{DFA}} = \{\langle B, w \rangle : B \text{ is a DFA that accepts input string } w\}$$

- The problem of testing whether a DFA B accepts an input string w is the same as the problem of testing whether $\langle B, w \rangle$ is a member of language A_{DFA}

Theorem 1

A_{DFA} is a decidable language.

Proof for the Last Theorem

Proof.

- Either construct a TM that decides A_{DFA} or
- Write a program (in any programming language) that simulates the work of the DFA B and checks if B accepts w or not



Theorem 2

A_{NFA} is a decidable language, where

$$A_{\text{NFA}} = \{\langle B, w \rangle : B \text{ is an NFA that accepts input string } w\}.$$

Theorem 2

A_{NFA} is a decidable language, where

$$A_{\text{NFA}} = \{\langle B, w \rangle : B \text{ is an NFA that accepts input string } w\}.$$

Theorem 3

A_{REX} is a decidable language, where

$$A_{\text{REX}} = \{\langle R, w \rangle : R \text{ is a R.E. that generates string } w\}.$$

- Consider **emptiness testing** for finite automata
- To test whether a finite automaton accepts any strings at all
i.e.,

$$E_{\text{DFA}} = \{ \langle A \rangle : A \text{ is a DFA and } L(A) = \emptyset \}$$

- Consider **emptiness testing** for finite automata
- To test whether a finite automaton accepts any strings at all
i.e.,

$$E_{\text{DFA}} = \{\langle A \rangle : A \text{ is a DFA and } L(A) = \emptyset\}$$

Theorem 4

E_{DFA} is a decidable language.

- Suppose there are two DFAs A and B
- Now consider a language that tests if both A and B recognize the same language i.e.,

$$EQ_{\text{DFA}} = \{\langle A, B \rangle : L(A) = L(B)\}$$

- Suppose there are two DFAs A and B
- Now consider a language that tests if both A and B recognize the same language i.e.,

$$EQ_{\text{DFA}} = \{\langle A, B \rangle : L(A) = L(B)\}$$

Theorem 5

EQ_{DFA} is a decidable language.

Decidability for Context Free Languages

Theorem 6

A_{CFG} is a decidable language, where

$$A_{\text{CFG}} = \{\langle G, w \rangle : G \text{ is a CFG that generates string } w\}.$$

Theorem 6

A_{CFG} is a decidable language, where

$$A_{\text{CFG}} = \{\langle G, w \rangle : G \text{ is a CFG that generates string } w\}.$$

Proof.

- If G is in Chomsky Normal Form, any derivation of a string w has $2n - 1$ steps, where $n = |w|$, so
- We create TM S for A_{CFG} as following:
- On input $\langle G, w \rangle$, convert G to an equivalent CNF form
- List all derivation with $2n - 1$ steps where $n = |w|$, except when $n = 0$ (in that case list all derivations of one steps)
- If any of these derivations generate w , *accept*, if not, *reject*.



Theorem 7

E_{CFG} is a decidable language, where

$$E_{\text{CFG}} = \{\langle G \rangle : G \text{ is a CFG and } L(G) = \emptyset\}.$$

Theorem 7

E_{CFG} is a decidable language, where

$$E_{\text{CFG}} = \{\langle G \rangle : G \text{ is a CFG and } L(G) = \emptyset\}.$$

Proof.

On input $\langle G \rangle$, where G is a CFG:

- Mark all terminal symbols in G
- Repeat until no new variables get marked:
 - Mark any variable A where G has a rule $A \rightarrow U_1 U_2 \cdots U_k$ and each symbol $U_1, \dots, U_k$ has already been marked
- If the start variable is not marked, *accept*; otherwise, *reject*



Theorem 8

EQ_{CFG} is not a decidable language, where

$$EQ_{CFG} = \{\langle G, H \rangle : L(G) = L(H)\}.$$

Theorem 8

EQ_{CFG} is not a decidable language, where

$$EQ_{CFG} = \{\langle G, H \rangle : L(G) = L(H)\}.$$

Theorem 9

Every context-free language is decidable.

Theorem 8

EQ_{CFG} is not a decidable language, where

$$EQ_{CFG} = \{\langle G, H \rangle : L(G) = L(H)\}.$$

Theorem 9

Every context-free language is decidable.

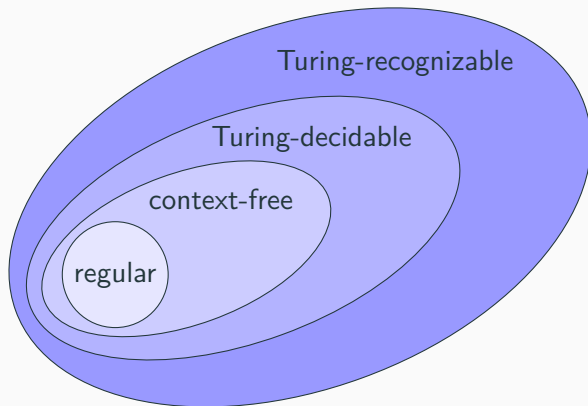
Proof.

Let G be a CFG for A and design a TM M_G that decides A . We build a copy of G into M_G . It works as follows:

$M_G =$ "On input w :

- Run TM S on input $\langle G, w \rangle$
- If this machine accepts, *accept*; if it rejects, *reject*."

Relationship among classes of languages



The Halting Problem

Unsolvable Problems

- Computers appear to be very powerful that we might believe that all problems will eventually be solvable
- Are there any unsolvable problems?
- What are those problems?
- Are these esoteric?
- It appears that some simple and ordinary problems are also unsolvable

Unsolvable Problems

- Suppose you are given a computer program and a precise specification of what that program is supposed to do (e.g., sort a list of numbers)
- You need to verify that the program performs as specified (i.e., it is correct)
- Since both the program and the specification are mathematically precise objects we might believe/hope that we can automate the process of verification by feeding this info into some computer program

Unsolvable Problems

- Suppose you are given a computer program and a precise specification of what that program is supposed to do (e.g., sort a list of numbers)
- You need to verify that the program performs as specified (i.e., it is correct)
- Since both the program and the specification are mathematically precise objects we might believe/hope that we can automate the process of verification by feeding this info into some computer program
- Unfortunately, general problem of software verification is not solvable by computers

Theorem 10

A_{TM} is undecidable, where

$$A_{\text{TM}} = \{\langle M, w \rangle : M \text{ is a TM and } M \text{ accepts } w\}.$$

Theorem 10

A_{TM} is undecidable, where

$$A_{\text{TM}} = \{ \langle M, w \rangle : M \text{ is a TM and } M \text{ accepts } w \}.$$

A_{TM} is called the **Halting Problem**

- Consider following Turing Machine U which recognizes A_{TM} .

- Consider following Turing Machine U which recognizes A_{TM} .

$U =$ “On input $\langle M, w \rangle$, where M is a TM and w is a string:

1. Simulate M on input w
2. If M ever enters its accept state, *accept*; if M ever enters its rejects state, *reject*”

- Consider following Turing Machine U which recognizes A_{TM} .

$U =$ “On input $\langle M, w \rangle$, where M is a TM and w is a string:

1. Simulate M on input w
 2. If M ever enters its accept state, *accept*; if M ever enters its rejects state, *reject*”
- U is called *universal Turing machine*

The Halting Problem

Definition 11

- Let A and B be two sets and a function f from A to B
- We say f is **one-to-one** if it never maps two different elements to the same place i.e., if $f(a) \neq f(b)$ whenever $a \neq b$
- We say that f is **onto** if it hits every element of B i.e., if for every $b \in B$ there is an element $a \in A$ such that $f(a) = b$
- We say A and B are the **same size** if there one-to-one, onto function $f : A \rightarrow B$
- A function that is both one-to-one and onto is called a **correspondence**.

The Halting Problem

- Let $\mathbb{N} = \{1, 2, 3, \dots\}$ and $\mathbb{E} = \{2, 4, 6, \dots\}$
- Both $\mathbb{N}$ and $\mathbb{E}$ have same size

The Halting Problem

- Let $\mathbb{N} = \{1, 2, 3, \dots\}$ and $\mathbb{E} = \{2, 4, 6, \dots\}$
- Both $\mathbb{N}$ and $\mathbb{E}$ have same size

Definition 12

A set is **countable** if either it is finite or it has same size as $\mathbb{N}$.

The Halting Problem: Countable Infinite Sets

- Set of integers is countable
- Set of odd numbers is is countable
- Set of even numbers is countable
- Set of rational numbers is countable :)

The Halting Problem: Countability of Set of Rational Numbers

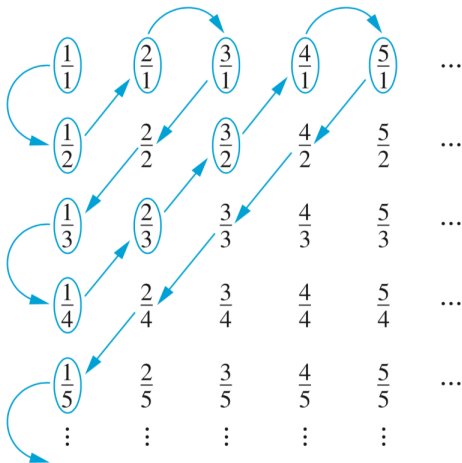
- We can list the positive rational numbers as a sequence

$$r_1, r_2, \dots, r_n, \dots$$

- Note that every positive rational number is the quotient $\frac{p}{q}$ of two positive integers
- We can arrange the positive rational numbers in a 2D grid by listing those with denominator $q = 1$ in the first row, those with denominator $q = 2$ in the second row, and so on
- First list the rational numbers p/q with $p + q = 2$, followed by $p + q = 3$ (shown in circles)
- Whenever we encounter a number p/q that is already in the list, don't list it (shown without circles)
- We can list all positive rational numbers once
- Therefore, set of rational number is countable

The Halting Problem: Countability of Set of Rational Numbers

Terms not circled
are not listed
because they
repeat previously
listed terms



The Halting Problem: An Uncountable Set

- The set of real numbers $\mathbb{R}$ is an uncountable set.

The Halting Problem: An Uncountable Set

- The set of real numbers $\mathbb{R}$ is an uncountable set.
- Assume $\mathbb{R}$ is countable, the subset $[0, 1]$ is also countable
- We can list real numbers between 0 and 1 in some order $r_1, r_2, r_3, \dots$, as (where $d_{ij} \in \{0, 1, 2, 3, \dots, 9\}$)

$$r_1 = 0.d_{11}d_{12}d_{13}d_{14} \dots$$

$$r_2 = 0.d_{21}d_{22}d_{23}d_{24} \dots$$

$$r_3 = 0.d_{31}d_{32}d_{33}d_{34} \dots$$

$$\vdots$$

- Now form a new decimal $r = 0.d_1d_2d_3d_4 \dots$, where $d_i = 4$ if $d_{ii} \neq 4$ and $d_i = 5$ if $d_{ii} = 4$

The Halting Problem: An Uncountable Set

- For example, suppose $r_1 = 0.23794102\dots$,
 $r_2 = 0.44590138\dots$, $r_3 = 0.09118764\dots$,
 $r_4 = 0.80553900\dots$, and so on
- We have $r = 0.d_1d_2d_3d_4\dots = 0.4544\dots$ where $d_1 = 4$,
 $d_2 = 5$, and so on
- r_1 is not equal to r_1 or r_2 or r_3 , and so on because r differs from the r_i in the i -th place to the right of decimal point for each i
- This means there is a real number $r \in [0, 1]$ which is not in the list
- Assumption is wrong and therefore $\mathbb{R}$ is uncountable

Corollary 13

Some languages are not Turing-recognizable.

Corollary 13

Some languages are not Turing-recognizable.

- **Proof Ideas**
- The set of all Turing machines is countable
- The set of all possible strings over some alphabet Σ is countable
- The set of all languages over Σ is uncountable

The Halting Problem

Theorem 14

A_{TM} is undecidable, where

$$A_{\text{TM}} = \{ \langle M, w \rangle : M \text{ is a TM and } M \text{ accepts } w \}.$$

The Halting Problem

Theorem 14

A_{TM} is undecidable, where

$$A_{\text{TM}} = \{\langle M, w \rangle : M \text{ is a TM and } M \text{ accepts } w\}.$$

- Assume A_{TM} is decidable
- Suppose H is a decider for A_{TM} i.e., on input $\langle M, w \rangle$, where M is a TM and w is a string, H halts and accepts if M accepts w , and H halts and rejects if M fails to accept w or,

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{if } M \text{ accepts } w \\ \text{reject} & \text{if } M \text{ does not accept } w. \end{cases}$$

The Halting Problem

- Now construct a new TM D with H as a subroutine
- On input $\langle M \rangle$, where M is a TM, D
 - Runs H on input $\langle M, \langle M \rangle \rangle$
 - Outputs the opposite of what H outputs, i.e., if H accepts, *reject*, and if H rejects, *accept*
- In summary,

$$D(\langle M \rangle) = \begin{cases} \textit{accept} & \text{if } M \text{ does not accept } \langle M \rangle \\ \textit{reject} & \text{if } M \text{ accepts } \langle M \rangle. \end{cases}$$

- Now consider this

$$D(\langle D \rangle) = \begin{cases} \textit{accept} & \text{if } D \text{ does not accept } \langle D \rangle \\ \textit{reject} & \text{if } D \text{ accepts } \langle D \rangle. \end{cases}$$

The Halting Problem

Use H to build a TM D that takes $\langle M, w \rangle$ as input

- H accepts $\langle M, w \rangle$ when M accepts w
- D rejects $\langle M \rangle$ when M accepts $\langle M \rangle$
- D rejects $\langle D \rangle$ when D accepts $\langle D \rangle$

Turing-Unrecognizable Language

- We know A_{TM} is undecidable but it is Turing-recognizable
- We will see a language is not even Turing-recognizable

Turing-Unrecognizable Language

- We know A_{TM} is undecidable but it is Turing-recognizable
- We will see a language is not even Turing-recognizable

co-Turing-recognizable

A language L is **co-Turing-recognizable** if it is the complement of a Turing-recognizable language.

Turing-Unrecognizable Language

- We know A_{TM} is undecidable but it is Turing-recognizable
- We will see a language is not even Turing-recognizable

co-Turing-recognizable

A language L is **co-Turing-recognizable** if it is the complement of a Turing-recognizable language.

Theorem 15

A language is decidable iff it is Turing-recognizable and co-Turing-recognizable.

Turing-Unrecognizable Language

- We know A_{TM} is undecidable but it is Turing-recognizable
- We will see a language is not even Turing-recognizable

co-Turing-recognizable

A language L is **co-Turing-recognizable** if it is the complement of a Turing-recognizable language.

Theorem 15

A language is decidable iff it is Turing-recognizable and co-Turing-recognizable.

- In other words, a language is decidable exactly when both it and its complement are Turing-recognizable

Corollary 16

$\overline{A_{\text{TM}}}$ is not Turing-recognizable.

Corollary 16

$\overline{A_{TM}}$ is not Turing-recognizable.

Proof.

- A_{TM} is Turing-recognizable
- If $\overline{A_{TM}}$ is also Turing-recognizable, A_{TM} would be decidable
- We know A_{TM} is not decidable, so $\overline{A_{TM}}$ must not be Turing-recognizable.

